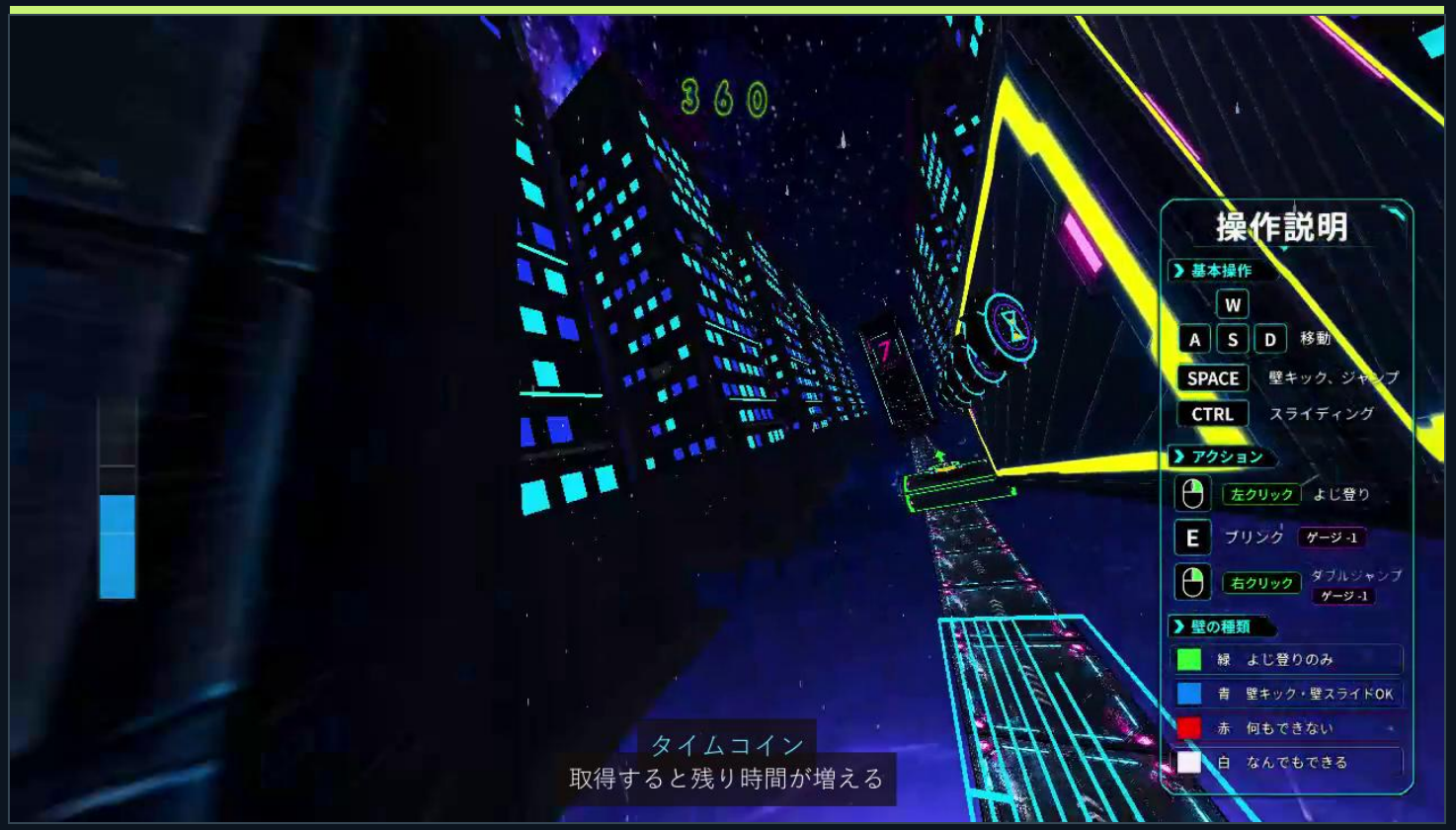
作品ポートフォリオ

ゲーム プログラマー

壁を蹴って進むアクションと、
作戦を組めるノードエディターを制作。

小菅 友暉

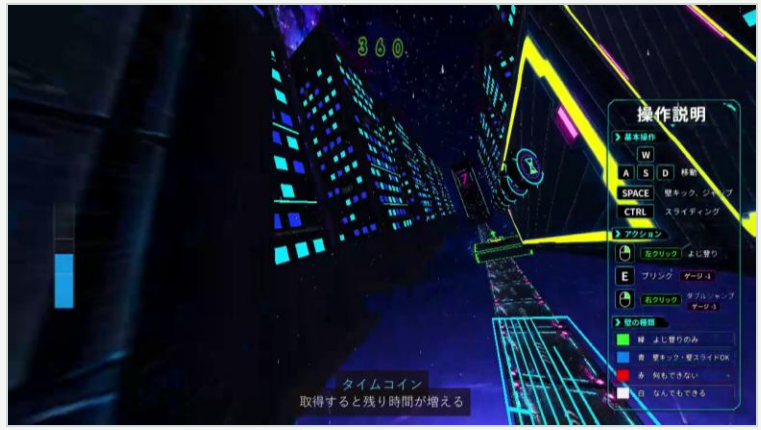


KICK LOOP / プレイ画面

C++ / DirectX 11 / C# / Unity

GitHubの作品一覧 ↗

主な制作作品



KICK LOOP

個人制作 / C++・DirectX 11

壁キックとジャンプの操作設計

P.03–09

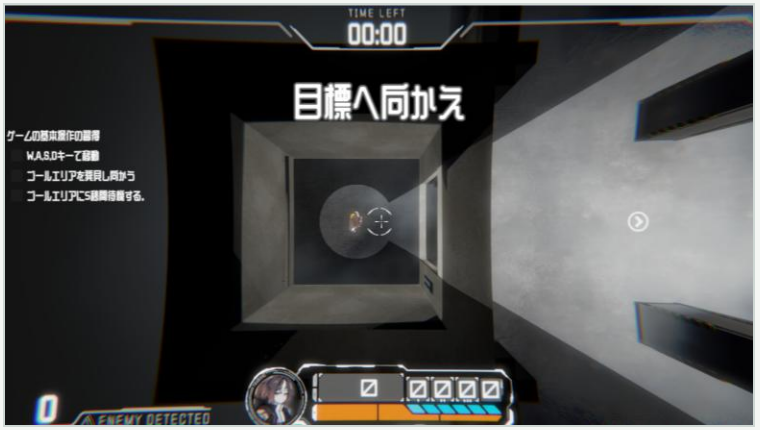


NodeBattle

個人制作 / C++・DirectX 11

ノードの編集・実行・保存

P.10–13



ANNiHILATION

チーム制作 / C#・Unity

担当　UIと画面ノイズの制御

P.14–16

プロフィール

HAL東京 ゲーム4年制学科

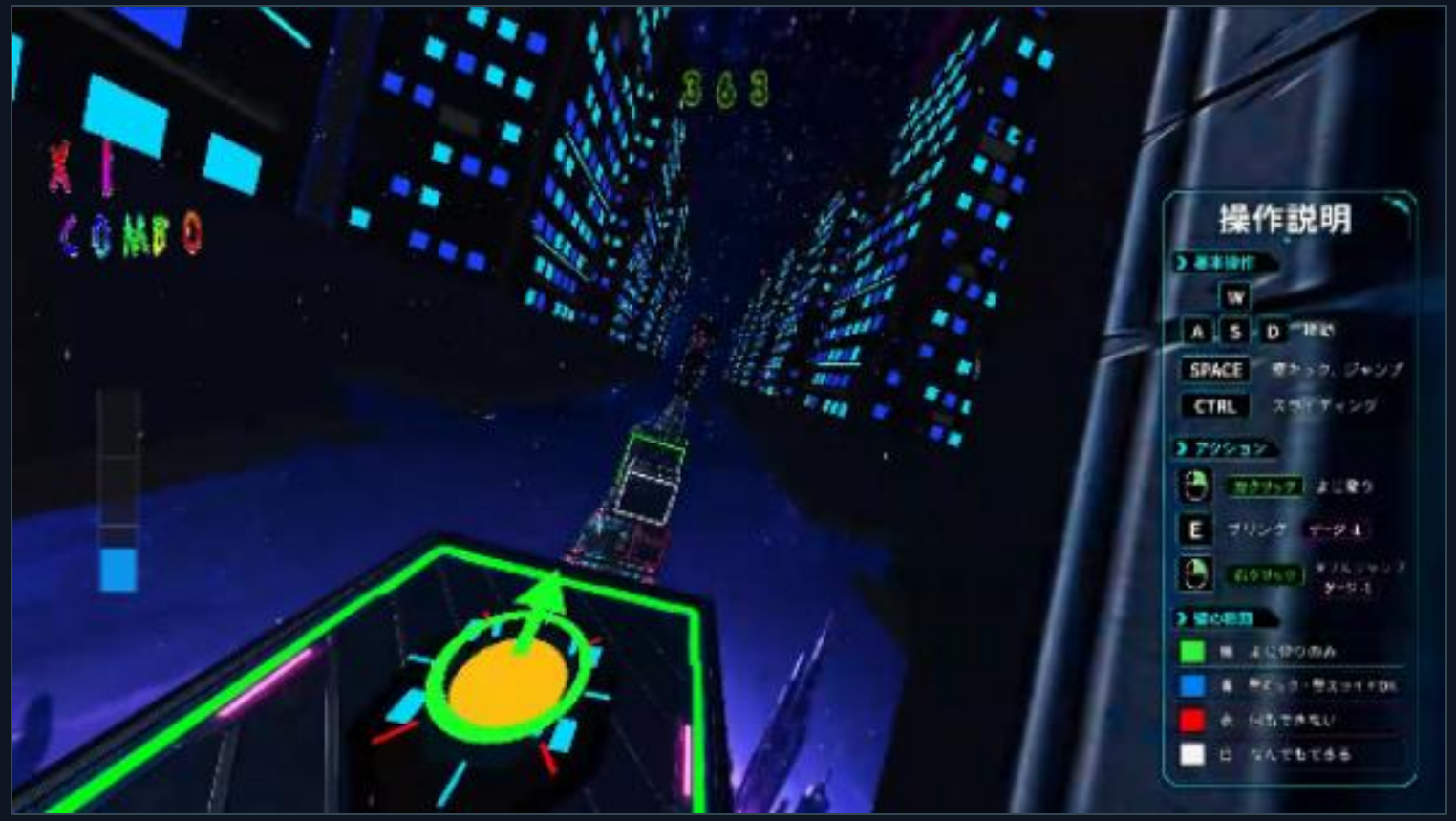
基本情報技術者試験 合格

CGエンジニア検定 ベーシック

HAL東京 HEW1年次 技術力賞

paiza スキルチェック Bランク

KICK LOOP



完成版のプレイ画面
タイトルのSTARTは視線で選択。ボタン面との交点で判定。

一人称パルクールアクション

壁キック、よじ登り、急降下を
つないで、制限時間内に
ゴールを目指すゲームです。

自作した主な処理

プレイヤー操作・区間生成
GPUパーティクル・音声管理

C++ / DirectX 11

完成版の動画 ↗　起動手順 ↗

STARTの選択処理 ↗　プロジェクトを見る ↗

複数の操作を、決まった順で反映する

ジャンプ、壁キック、急降下が同じフレームで速度を変えるため、入力の判定と移動の確定を分けました。

Controller

入力と接地・壁の状態を確認。
各アクションが変更を要求する。

→

ActionRequest

速度や行動の要求を
そのフレームの分だけまとめる。

→

Resolver

要求を受け取り、移動・重力・
衝突・接地の結果を反映する。

壁キック時の流れ

壁側の処理が上向きの速度を要求。
移動の確定はResolverに任せるため、
接地や衝突との処理順を一か所で追えます。

playerWallKickController.cpp

```
action.SetVerticalVelocity(wallKickParameter.jumpPower);
```

player.cpp ↗　playerActionResolver.cpp ↗　壁キックのコード ↗

同じSpaceキーで、壁キックとジャンプを扱う

壁キックに使った入力が通常ジャンプにも残ると、着地直後に意図しないジャンプにつながります。

空中で壁キックが可能

壁に触れている間と、壁を離れた直後は
通常ジャンプ側の入力を消します。

着地前にジャンプを入力

通常ジャンプの入力を最大8フレーム保持し、
接地した時点で残っていれば使います。

playerJumpController.cpp | 壁に接触している場合

```
if (state.isTouchWall)
{
    m_JumpBufferFrame = 0;
    return;
}
```

空中かつSpaceの押下を検出したときの分岐です。

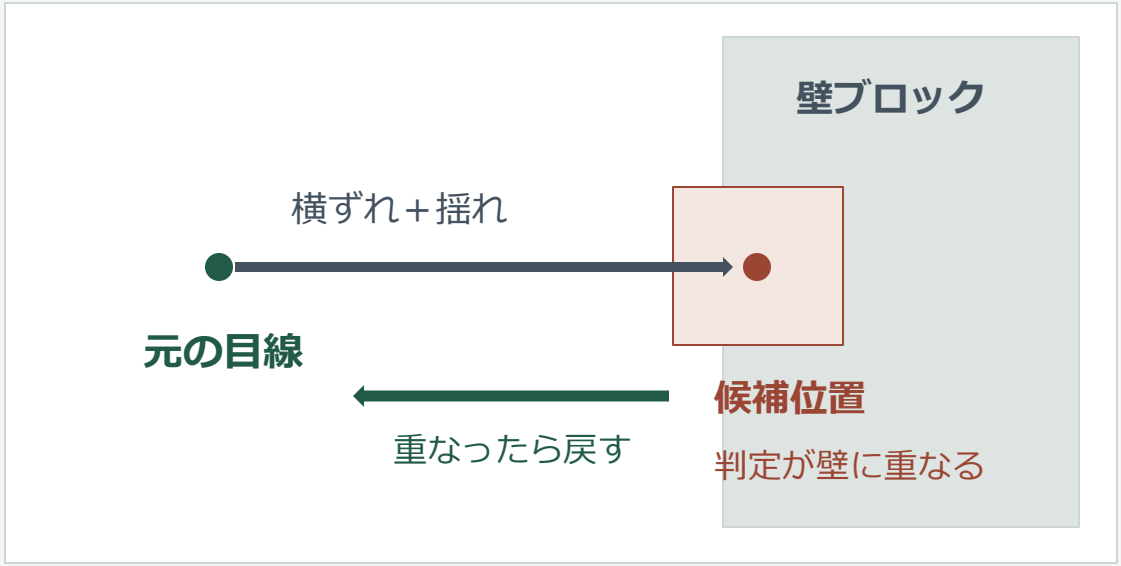
入力を保持・消去する処理 ↗

ジャンプの初期設定 ↗

壁に重なるカメラを、元の目線へ戻す

壁滑りの横ずれや揺れを加えると、プレイヤーが壁の手前にいても、視点が壁に入り込むことがあります。

上から見たカメラ位置



位置を戻したフレームでも、視線の向きや傾きは反映します。

```
camera.cpp  |  Camera::Update

Vector3 desiredPosition = basePosition + m_ShakeOffset;

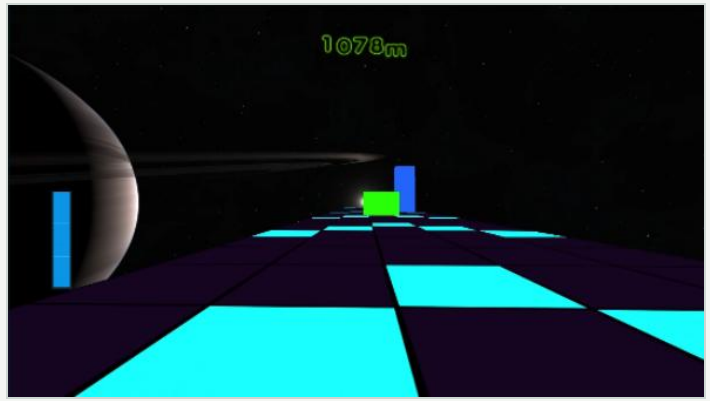
if (IsCameraHitBlockByPlayerHit(desiredPosition,
    player->GetHitSize()))
{
    m_Position = playerEyePosition;
}
else
{
    m_Position = desiredPosition;
}
```

候補位置に高さの薄い当たり判定を置き、
横幅と奥行きにはプレイヤーの判定より少し余裕を持たせます。

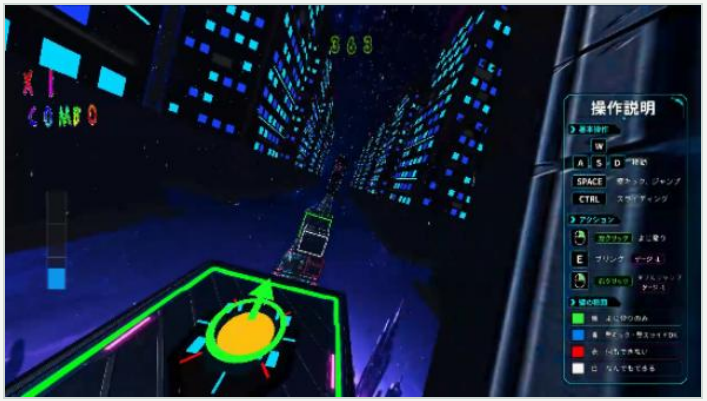
カメラ位置の決定 ↗　　**ブロックとの重なり判定** ↗

床の設定はJSON、区間の配置はC++

床の見た目を調整するデータと、進行に合わせて出す障害物の配置を分けています。



試作版



完成版

床の設定

位置、サイズ、床タイルや色を
JSONのGroundから読み込みます。

区間の配置

壁・障害物・コインの配置はC++で定義。
プレイヤーの前方に次の区間を出します。

障害物を再利用

通過した障害物は更新と描画を止めてプールに戻します。
次の配置では状態をリセットして使い、足りない分だけ生成します。

床のJSON ↗　区間の定義 ↗　生成と回収 ↗　試作版の動画 ↗

ゴール演出で、通過したコースを復元する

プレイ中は通過したブロックを回収。ゴール後にカメラがコースを映し直すときは、区間の記録から作り直します。

プレイ中に記録

生成した区間の種類と
配置した距離を残す。

→

カメラの周辺を検索

記録から、カメラの前後に
重なる区間を探す。

→

必要なブロックを復元

区間の定義を使って元の位置に配置。
復元済みの区間は飛ばす。

ゴール演出中の復元範囲

模式図



カメラの位置　この範囲に重なる区間のブロックを復元

区間の記録 ↗　ブロックの復元 ↗　ゴール演出の開始 ↗

粒子の移動と寿命をGPUで更新する

CPUから発生範囲やパラメーターを渡し、コンピュートシェーダーで粒子ごとの状態を進めます。



プレイ中の粒子演出

粒子ごとに行う処理

速度・位置を更新し、残り寿命が尽きた粒子を無効にします。

gpuParticleCS.hlsl | 更新式の抜粋

```
particle.VelocityMaxLife.y += gravity * deltaFrame;
    // ...
particle.PositionLife.w -= deltaFrame;
```

粒子更新のコード ↗

配布版とソースの動作

配布版は音声付きで、Visual Studio不要です。
ソースからのビルドにはCRI ADX LE SDKが必要です。

ビルド手順 ↗　　**CRI SDKの配置手順 ↗**

NodeBattle

ビジュアルプログラミング

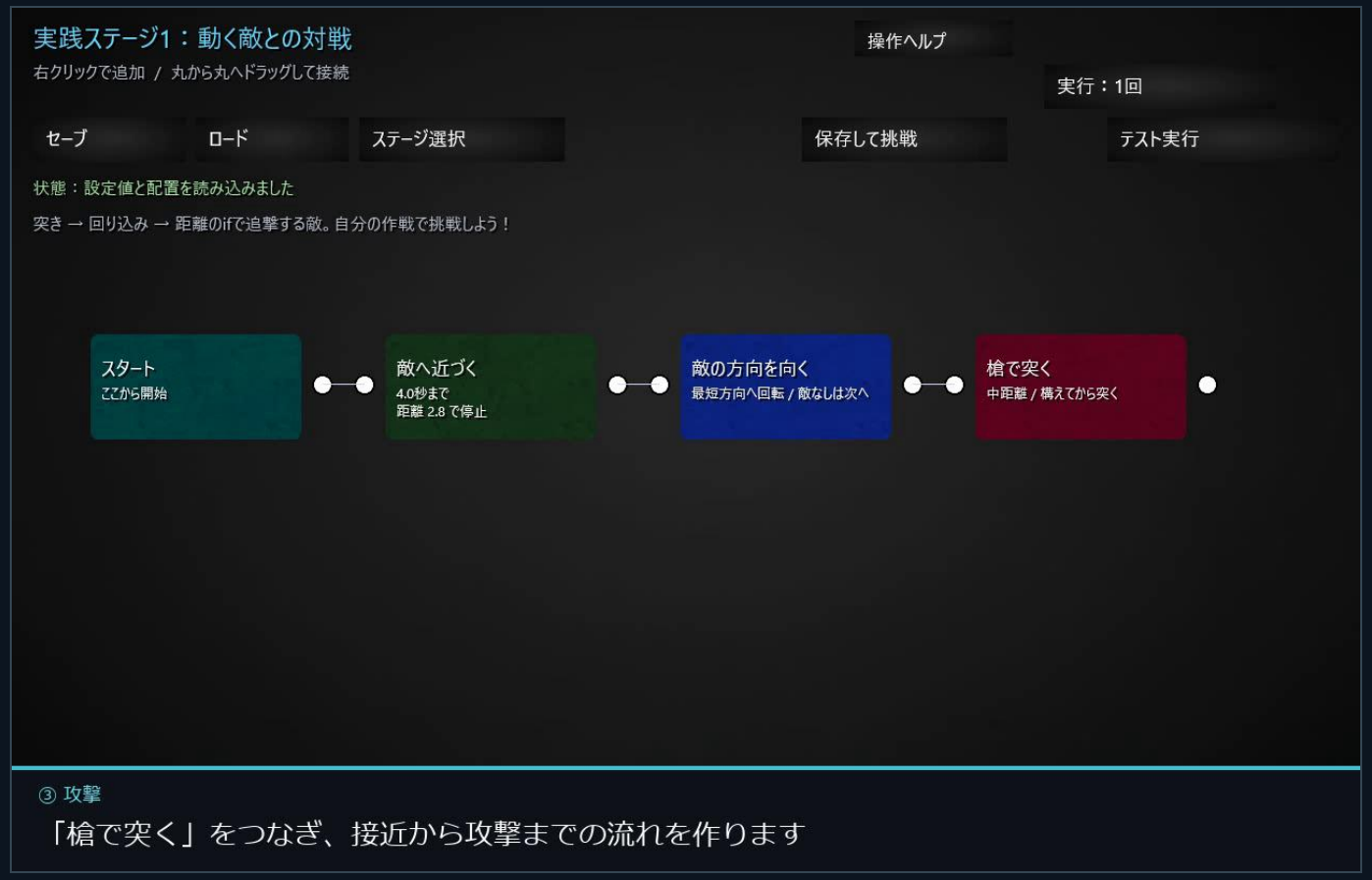
移動・条件・攻撃をつなぎ、
組んだ作戦をバトルで試せます。

自作した主な処理

ノードの編集と接続チェック
行動の実行と通過した経路の表示
作戦のJSON保存と敵の行動設定

C++ / DirectX 11

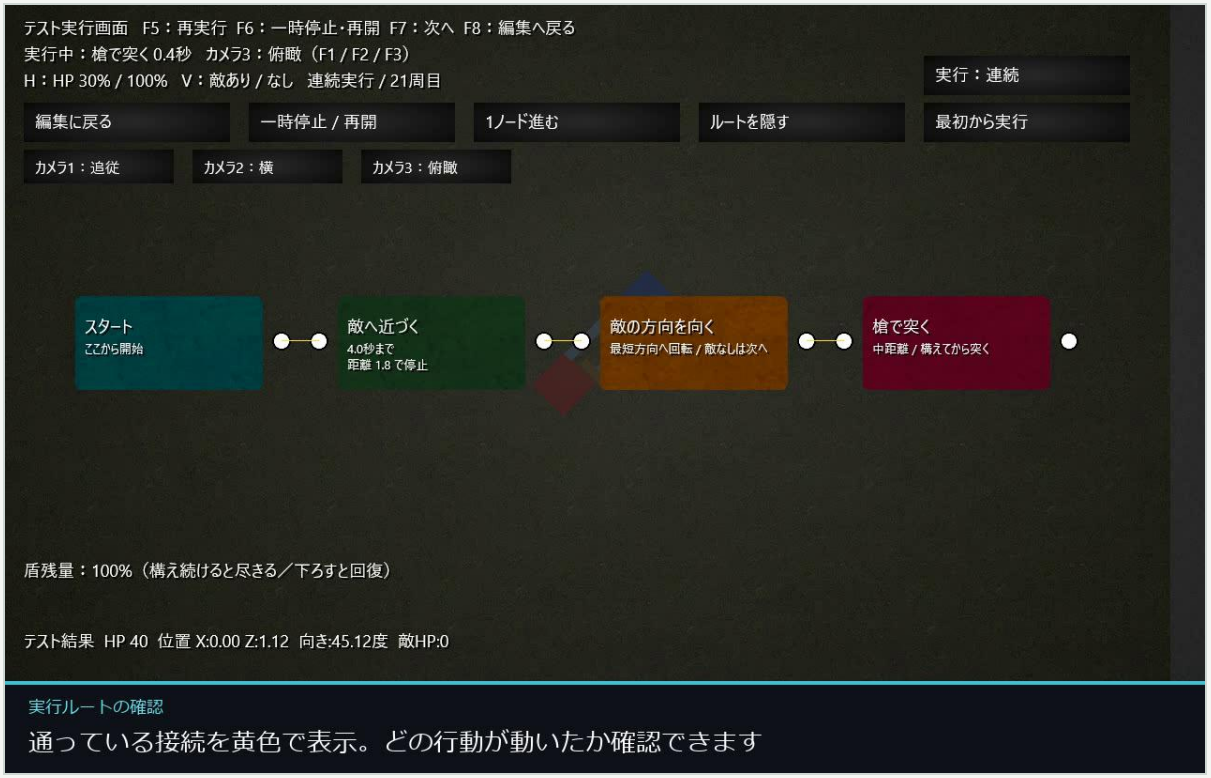
操作動画 ↗　**プロジェクト ↗**



ノードをつないで作戦を組む画面

ノードが終わってから、次の行動へ進む

移動や攻撃をフレームごとに進め、通った分岐を画面上で確かめられるようにしました。



テスト画面では、通過した接続を黄色で表示します。

Begin	行動の開始時に1回呼ぶ。
Tick	毎フレーム進め、完了・失敗を判定する。
End	行動を終了する。完了したら次のノードへ。

開始と条件はExecuteで次を決定。
一時停止と1ノードずつの実行にも対応。

実行・経路の記録 ↗　　**経路の描画** ↗

接続ミスを見つけて、作戦を保存する

接続時、実行前、読み込み時に、それぞれ必要な条件を確認しています。

接続時

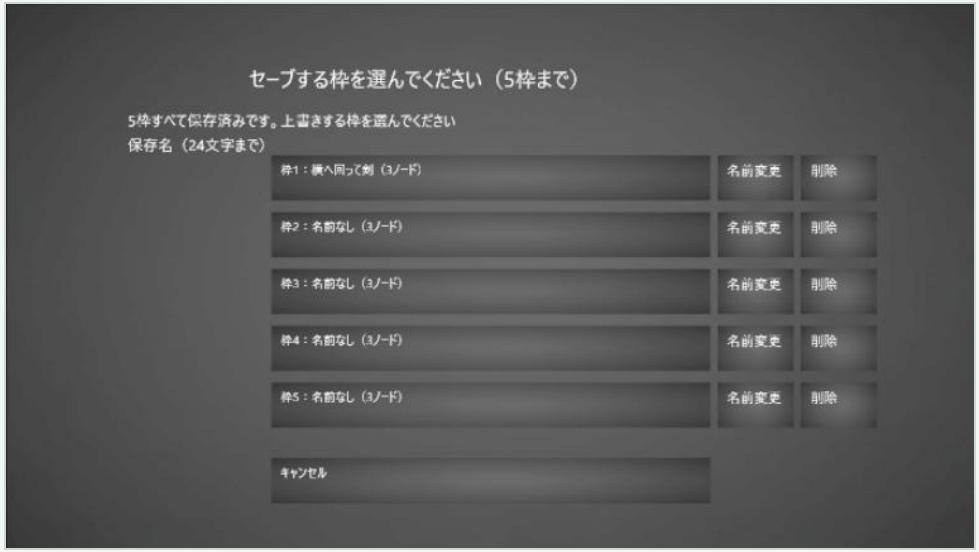
自己接続や循環を作る線は登録しない。

実行前

開始点や分岐の接続漏れを確認し、理由を表示する。

読み込み時

保存したIDから接続を復元し、検証後に編集中の作戦を置き換える。



作戦は名前付きで5枠まで保存できます。

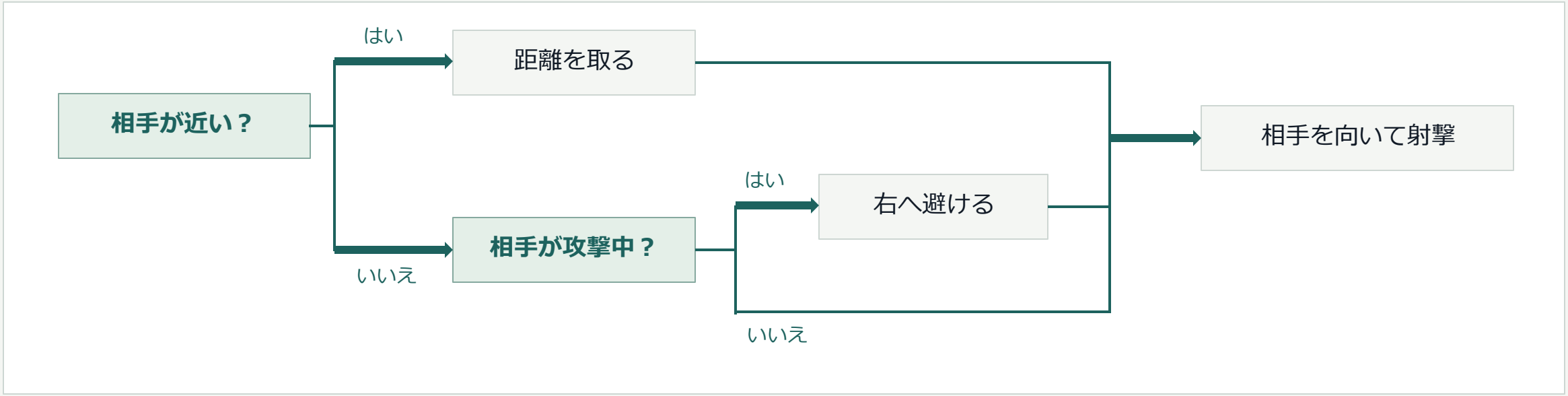
JSONにはノードの種類、ID、設定、配置、接続を記録。保存は一時ファイルを書き終えてから置き換えます。

接続時の検証 ↗ **実行前の検証 ↗** **JSONの復元 ↗** **セーブ処理 ↗**

敵の作戦も、同じノード実行処理で動かす

ステージごとのJSONから敵のHPと行動ノードを読み込み、プレイヤーと同じNodeRuntimeで実行します。

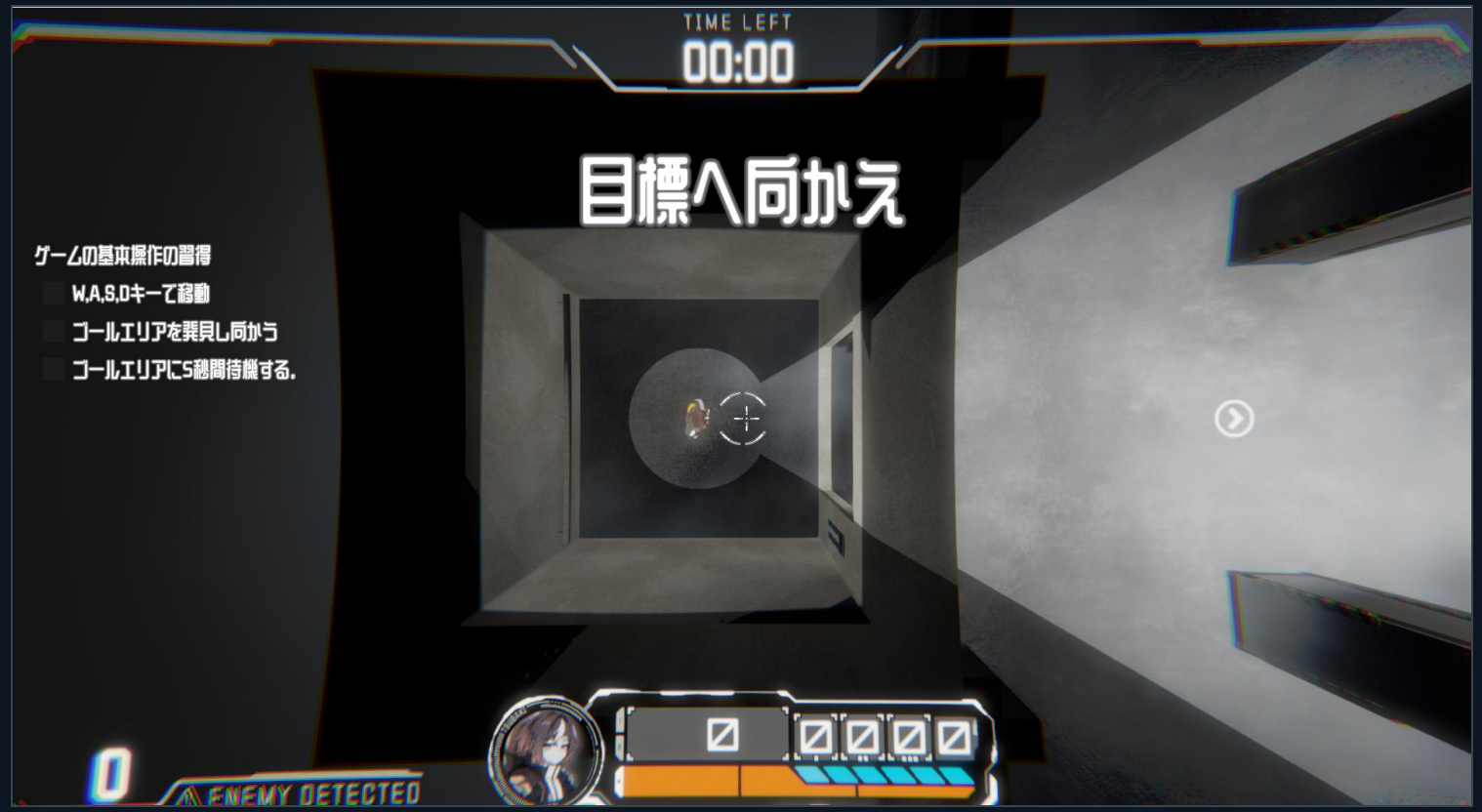
実践ステージ2「距離を取る射撃手」



射撃後はガードと待機に進み、作戦を繰り返します。

敵の作戦データ ↗　　**ステージごとの読み込み処理** ↗

ANNiHILATION



画面下部に、操作中に必要な情報を表示。

担当したUIと画面演出

HP・武器・アビリティゲージ
画面外の目標を示すマーカー
ダメージ・ゲームオーバー時のノイズ

壁を壊したり作ったりしながら、
敵を避けてゴールを目指すゲーム。

C# / Unity 6000.3.11f1 / URP

担当コード一覧 ↗

残量はゲージに、画面外の目標は画面端に

回復の途中と、目標が見切れたときの方向がプレイ中に分かるようにしました。

アビリティゲージ



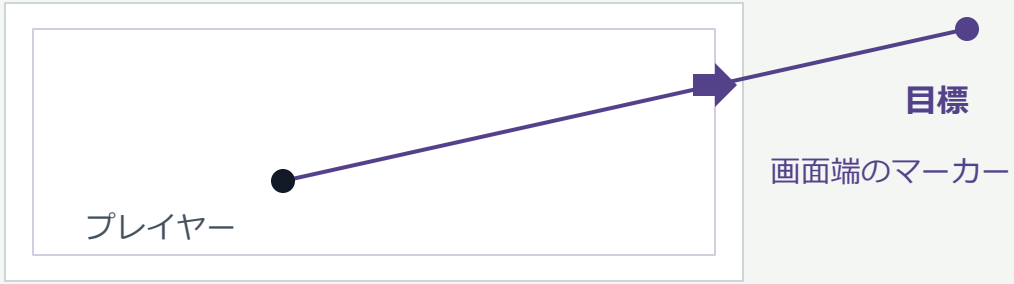
残量から各枠の番号を引き、0〜1に収めます。
たとえば残量2.5なら、3枠目を半分まで塗ります。

```
UI_AbilityGauge.cs

float capsuleFill = Mathf.Clamp01(currentEnergy - i);
m_capsuleImages[i].fillAmount = capsuleFill;
```

ゲージの更新処理 ↗

目標マーカー



目標へ向かう線が、余白を取った枠と
最初に交わる位置にマーカーを置きます。

```
UI_GoalNavi.cs

float t = Mathf.Min(toEdgeX, toEdgeY);
Vector2 navPos = playerScreenPos + dir * t;
```

マーカーの位置計算 ↗

世界観に合わせ、被弾をノイズで伝える

チームで相談し、当初の赤いビネット案からノイズへ変更。イベントに応じて強さを制御しました。



作品のタイトル画面

プレイヤーの被弾・死亡イベント

被弾

一時的にノイズを出し、時間とともに弱める。

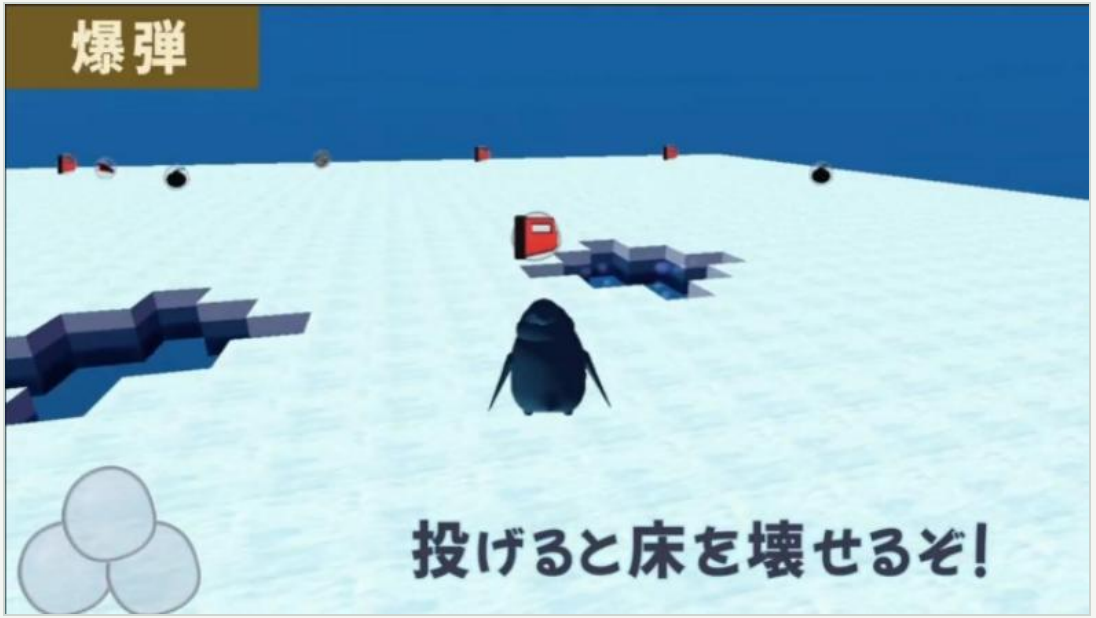
ゲームオーバー

ノイズを徐々に強めて、状態の変化を伝える。

制御側で決めた強さをRenderer Featureで読み取ります。

ノイズ制御の担当コード ↗　　**強さを読み取る箇所 ↗**

ほかの制作経験



DynamicBreaker

チーム制作 / C++・DirectX 11

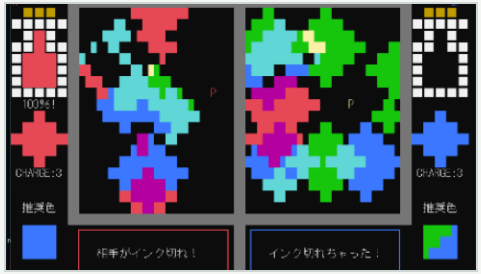
担当　床の配置・生成、シャチの移動・当たり判定



タイムアタックレーシング

個人制作 / 1週間 / C++・DirectX 11

車両制御、地形生成、
追従カメラ、3D音響。



SPLASTER

個人制作 / 1か月 / C

2人対戦の進行管理と
コンソールUI。

担当コード ↗　　プレイ動画 ↗　　ICEの起動手順 ↗　　その他の作品フォルダー ↗

WORK LINKS

作品リンク

github.com/Koyu0729/project

KICK LOOP

個人制作 / C++・DirectX 11

プレイ動画 ↗ | アクションのコード ↗ | 起動手順 ↗

NodeBattle

個人制作 / C++・DirectX 11

操作動画 ↗ | 実行処理のコード ↗ | 起動手順 ↗

ANNiHILATION

チーム制作 / UI・ノイズ制御

担当コード一覧 ↗ | 画面ノイズの制御 ↗

DynamicBreaker

チーム制作 / 床とシャチ

プレイ動画 ↗ | シャチのコード ↗ | 起動手順 ↗

動画がプレビューされない場合は、リンク先で「View raw」を選んでください。

最後までお読みいただき、ありがとうございました。